

AI Philosophy: A Design Discipline for Systems Architecture

Addressing the Structural Causes of Enterprise AI Failure

James (JD) Longmire

Northrop Grumman Fellow (unaffiliated research)

ORCID: 0009-0009-1383-7698

Correspondence: jdlongmire@outlook.com

Abstract

Enterprise AI investment of \$30-40 billion is producing a 95% failure rate. Operational explanations cannot account for why the same failure patterns converge across industries, vendors, and use cases. This paper argues that the failures are structural and establishes AI Philosophy as a design discipline to address them. The discipline operates across three axes: horizontal (infrastructure and data), vertical (epistemology and reasoning quality), and grounding (teleology and mereology). Current approaches optimize along the horizontal axis while neglecting the grounding considerations that determine production success. We present the business case, diagnose why current approaches fail, introduce the three-axis framework, develop a research agenda, and demonstrate practical architectural application. AI Philosophy is not philosophy applied to AI after the fact. It is a design discipline: the missing piece that explains why mature programs fail and what to do about it.

1. Introduction

Something is wrong with enterprise AI, and the usual explanations do not account for it.

The investment is massive. The technology is genuine. The engineering talent is world-class. Yet 95% of enterprise AI pilots fail to reach production, and those that do frequently fail to deliver measurable business value. The pattern holds across industries, across vendors, across use cases. Organizations with excellent infrastructure and clean data experience the same failures as those still building foundations.

The standard explanations point to operational causes: brittle workflows, weak contextual learning, misalignment with day-to-day operations, insufficient change management, unrealistic expectations. These explanations are not wrong, but they are incomplete. They describe symptoms without identifying the underlying condition. If the causes were purely operational, we would expect variation. Organizations with better operations would succeed more often. The convergence of failure across diverse contexts suggests something structural is being missed.

This paper argues that the missing element is grounding. Current AI architecture addresses infrastructure (getting the right information to the right place) and, increasingly, epistemology (whether systems reason well about what they receive). But almost no architectural work addresses the grounding questions: What is this system for? How does it relate to larger wholes? Without answers to these questions, systems can be efficient, well-sourced, and epistemically calibrated while still serving no good end-or while working perfectly in isolation and failing catastrophically in integration.

Philosophy is not typically considered a design discipline. It should be. The questions that philosophy addresses-questions of purpose, knowledge, and part-whole relationships-are precisely the questions that current AI architecture ignores and that the 95% failure rate suggests we cannot afford to keep ignoring.

This paper establishes AI Philosophy as a design discipline for systems architecture. It provides a framework for diagnosing structural failures, a research agenda for systematic development, and practical guidance for architectural application. The goal is not to add philosophical sophistication as a luxury. It is to address the structural causes of failure that operational interventions cannot reach.

2. The Business Case: Risk, Cost, and Opportunity

2.1 The Scale of Failure

In July 2025, MIT's Project NANDA released "The GenAI Divide: State of AI in Business 2025," a comprehensive study based on 52 executive interviews, surveys of 153 senior leaders, and analysis of over 300 public AI deployments. The headline finding was stark: 95% of enterprise AI pilots deliver no measurable impact on profit and loss. Only 5% of integrated AI systems create significant business value.

This is not a small-sample anomaly. The finding converges with other data points. S&P Global reported that 42% of companies abandoned most of their AI initiatives in 2024, up dramatically from 17% the previous year. The average organization scrapped 46% of AI proof-of-concepts before they reached production. Boston Consulting Group's 2024 analysis of 1,000 executives across 59 countries found that only 26% of companies have developed capabilities to move beyond proof of concept, while a mere 4% consistently generate significant AI value. Gartner predicts that 30% of generative AI projects will be abandoned after proof of concept by the end of 2025.

The financial stakes are substantial. Individual pilot failures cost between \$500,000 and \$2 million, with complex implementations reaching \$5 million or more. Aggregate enterprise spending on generative AI has reached \$30-40 billion, with the vast majority producing no measurable return.

2.2 Why Operational Explanations Are Insufficient

The MIT NANDA report, like most analyses, attributes failures to operational causes: systems don't fit into real workflows, they can't remember or adapt, there's misalignment with day-to-day operations. These explanations are accurate as far as they go. But they raise a question they cannot answer: why do the same patterns repeat?

If the failures were primarily operational, we would expect significant variation based on operational maturity. Organizations with sophisticated change management, clear workflows, and strong technical teams should succeed at higher rates. Yet the failure pattern holds across organizational types. Large enterprises run the most pilots but convert the fewest. Mid-market organizations move faster from pilot to production but still experience high failure rates. The

consistency suggests that operational execution, while necessary, is not the determining factor.

The convergent failure pattern points to structural causes-causes that persist regardless of operational quality because they exist at the architectural level. Organizations are solving the wrong problem. They are optimizing infrastructure and operations while leaving structural questions unaddressed.

2.3 The Cost of Continued Horizontal Investment

Organizations are investing heavily in what this paper calls the horizontal axis: infrastructure, retrieval systems, context management, scaling, cost optimization. This investment is necessary. Without solid infrastructure, nothing else works. But the failure data suggests that horizontal investment alone does not produce success.

The MIT report found that general-purpose tools like ChatGPT and Copilot are widely deployed-over 80% of organizations have explored or piloted them-but impact is limited to individual productivity rather than enterprise transformation. Enterprise-grade systems fare worse: 60% of firms evaluated them, but just 20% reached pilot stage and only 5% went live.

The pattern indicates a ceiling. Horizontal improvements produce horizontal gains: faster retrieval, cleaner data, more efficient scaling. But horizontal gains do not translate into production success when the structural issues remain unaddressed. Organizations continue investing in the dimension they understand while the failures continue for reasons their framework cannot articulate.

2.4 What Solving This Is Worth

The arithmetic is straightforward. If the current success rate is 5%, moving to 20% represents a fourfold improvement in return on AI investment. For an organization spending \$10 million annually on AI initiatives, that shift represents the difference between \$500,000 in value creation and \$2 million.

But the value extends beyond direct returns:

Reduced waste. Framework-informed design catches structural failures before deployment. The cost of identifying a grounding problem at the design stage is a fraction of the cost of discovering it after a failed pilot.

Faster time to value. Systems designed with clear purpose and part-whole coherence reach production faster because they don't require the iterative rework that unclear purpose produces.

Integration success. The agent failure pattern-components that work in isolation failing when integrated-is a mereological failure. Systems designed for part-whole coherence don't exhibit this pattern at the same rates.

Diagnostic capacity. Organizations that can identify which axis a failure belongs to can respond appropriately. Horizontal failures need horizontal solutions. Vertical failures need vertical solutions. Grounding failures need grounding solutions. Without the framework, all failures look operational, and operational interventions are applied to structural problems.

2.5 The Competitive Stakes

The 95% failure rate is an industry-wide phenomenon, but it will not remain evenly distributed. Organizations that develop the capacity to address structural causes will separate from those that continue applying operational solutions to structural problems.

Early adoption creates compounding advantage. Organizations that learn to diagnose grounding failures accurately will make better investment decisions. Those that build architectural capacity for epistemic self-awareness, teleological clarity, and mereological coherence will experience higher success rates. The learning curve favors early movers.

The risk of inaction is continued investment in approaches with documented 95% failure rates. The opportunity is developing a structural capacity that competitors lack.

3. Why Current Approaches Fail: A Diagnostic

The business case establishes that something structural is being missed. This section examines what has been tried and why it does not address the structural problem.

3.1 The Horizontal-Only Trap

Current AI discourse lives primarily on the horizontal axis. The major concerns are infrastructure: retrieval-augmented generation, vector databases, context window management, fine-tuning strategies, prompt engineering, cost optimization, scaling. These are legitimate engineering concerns. Real work gets done here, and organizations that neglect infrastructure will fail for infrastructure reasons.

But infrastructure is necessary, not sufficient. The 95% failure rate persists in organizations with excellent infrastructure. The MIT report notes that enterprises run the most pilots but convert the fewest-not because their infrastructure is worse, but because infrastructure excellence does not address the structural causes of failure.

The horizontal-only trap is mistaking infrastructure for the whole problem. When every failure looks like an infrastructure problem, every solution looks like an infrastructure investment. Organizations caught in this trap respond to structural failures with horizontal interventions, producing expensive non-solutions.

3.2 Technical Epistemology Without Architectural Integration

The vertical axis-epistemology and reasoning quality-is receiving increasing attention. Research on uncertainty quantification and confidence calibration has accelerated, with growing work on methods for estimating how certain language models are about their outputs.

More fundamentally, multiple independent research teams have established that hallucination is mathematically inevitable. Xu, Jain, and Kankanhalli (2024) proved using learning theory that large language models cannot learn all computable functions and will therefore inevitably hallucinate when used as general problem solvers. Banerjee and colleagues (2024) demonstrated that hallucinations stem from the fundamental mathematical and logical structure of these systems, drawing on computational theory and Gödel's First Incompleteness Theorem. OpenAI's own research (2025) identified three mathematical factors making hallucinations inevitable: epistemic uncertainty when information appears rarely in training data, model limitations where tasks exceed architectural capacity, and computational intractability where even superintelligent systems could not solve certain problems.

This research establishes architectural limits. Hallucination is not an engineering problem awaiting a sufficiently clever solution. It is a structural feature of how these systems work.

Yet this insight has not translated into architectural practice. Calibration is treated as a training objective or a post-hoc evaluation metric, not as an architectural feature. Systems are trained to be calibrated, then evaluated on calibration, but they are not structured to track their own epistemic status. The difference matters. A system trained to express appropriate confidence is not the same as a system that knows what it knows. The former can be manipulated by training distribution; the latter requires architectural capacity for epistemic self-awareness.

The gap is between technical epistemology and architectural integration. The research establishes what systems cannot do. The architectural implications-what systems need to be structured to do instead-remain largely undeveloped.

3.3 Alignment Philosophy Without Teleological Foundations

The AI alignment community has produced substantial philosophical work. Zhi-Xuan and colleagues (2025), publishing in *Philosophical Studies*, challenged the dominant "preferentist" approach to alignment. They argued that preferences fail to capture the "thick semantic content" of human values and that utility representations neglect the possible incommensurability of those values. Gabriel and Keeling (2025), also in *Philosophical Studies*, developed principle-based approaches to alignment as the fair treatment of claims.

This work is valuable. It addresses genuine problems in how AI systems are trained to behave. But it focuses on a specific question: given that we are building AI systems, what values should they embody, and how do we encode those values reliably?

That question presupposes an answer to a prior question: what is this system for? Alignment assumes purpose. A system cannot be aligned with human values in the abstract; it must be aligned for some function, some role, some place in human activity. The preferentist critique correctly notes that values are "thick"-embedded in contexts of meaning. But those contexts include purposes. What the system is for shapes which values are relevant and how they apply.

Current alignment work operates downstream of the teleological question. It takes purpose as given and asks how to align behavior with values within that

purpose. The structural question-what determines purpose, how purpose constrains design, what happens when purpose is unclear-remains largely unaddressed.

A system can be perfectly aligned with human values and still fail in production if its purpose is unclear or if it is integrated into a larger system in ways that create incoherence. Alignment is necessary but not sufficient. The grounding questions remain.

3.4 The Missing Mereological Analysis

Mereology-the study of parts and wholes-has a rich theoretical development in philosophy. Work by Cotnoir and Varzi (2021) provides comprehensive treatment of parthood relationships. Recent applications have connected mereological concepts to emergence and complex systems. McQueen and Tsuchiya (2023) explored integrated information as a restriction on mereological composition.

None of this work has been applied to AI systems architecture.

The absence is striking because the dominant failure pattern is mereological. The MIT report describes systems that work in demos but fail in production, agents that perform well in isolation but fail when integrated into workflows, pilots that succeed on their own terms but deliver no P&L impact. These are part-whole failures. The parts work; the wholes do not.

Current architectural discourse has no vocabulary for this. Failures are attributed to "integration challenges" or "workflow misalignment"-operational terms that describe symptoms without identifying the structural cause. The structural cause is that systems are designed as parts without consideration of the wholes they will belong to. They are not designed to model their own boundaries, to represent downstream use, or to evaluate their outputs against system-level success criteria.

Mereological coherence-the property of parts working correctly within wholes-is not an architectural concern in current practice. It should be. The high failure rate is, substantially, a mereological failure rate.

3.5 The "Philosophy Eats AI" Turn and Its Limits

In January 2025, MIT Sloan Management Review published "Philosophy Eats AI" by Schrage and Kiron. It became the publication's most-read article of the year.

The authors argued that philosophy shapes AI through teleology (what models should achieve), epistemology (what counts as knowledge), and ontology (how AI represents reality). They claimed that privileging ethics undervalues philosophy's true impact and that organizations failing to cultivate philosophical insight would fail to realize returns on AI investment.

The piece opened a conversation. It signaled to a business audience that philosophy matters for AI in ways beyond ethics. But it did not develop the philosophical content. Critics noted that the authors "fail to clarify their own working assumptions" and "don't seem willing to actually get at any of the deeper philosophical questions."

The space was opened. It has not been filled. The recognition that philosophy matters has not translated into a systematic treatment of how philosophy applies to AI architecture. What would teleologically-informed design look like? How does epistemology become an architectural feature? What are the mereological requirements for systems that integrate into larger wholes?

These questions require development. This paper provides it.

3.6 The Gap This Paper Fills

The landscape shows scattered contributions without integration:

- Business recognition that philosophy matters, without philosophical development
- Technical work on calibration and uncertainty, without architectural integration
- Alignment philosophy focused on values, without teleological foundations
- Rich mereological theory, never applied to AI systems
- Agent failures explained operationally, never structurally

No existing work systematically connects these domains. No one has proposed AI Philosophy as a design discipline with defined scope, method, and research agenda spanning the three axes.

The framework this paper provides is the missing integration layer. It connects the business problem (95% failure rate) to the structural causes (grounding failures) to the disciplinary response (AI Philosophy) to the practical application

(framework-informed architecture). The scattered contributions find their place within a unified treatment.

4. The Three-Axis Framework

The failures documented in Section 2 and the diagnostic gaps identified in Section 3 point toward a framework. AI architecture requires attention to three axes, only one of which receives systematic treatment in current practice.

4.1 Overview

The three axes address different questions with different methods. All three are required for production success.

The horizontal axis asks: Is the right information getting to the right place efficiently? This is infrastructure.

The vertical axis asks: Does the system reason well about what it receives, and does it know when it doesn't? This is epistemology.

The grounding axis asks: What is this system for, and how does it fit into larger wholes? This is teleology and mereology.

Current practice treats the horizontal axis as the whole problem. Sophisticated practitioners are beginning to address the vertical axis. Almost no one addresses the grounding axis. The 95% failure rate reflects this imbalance.

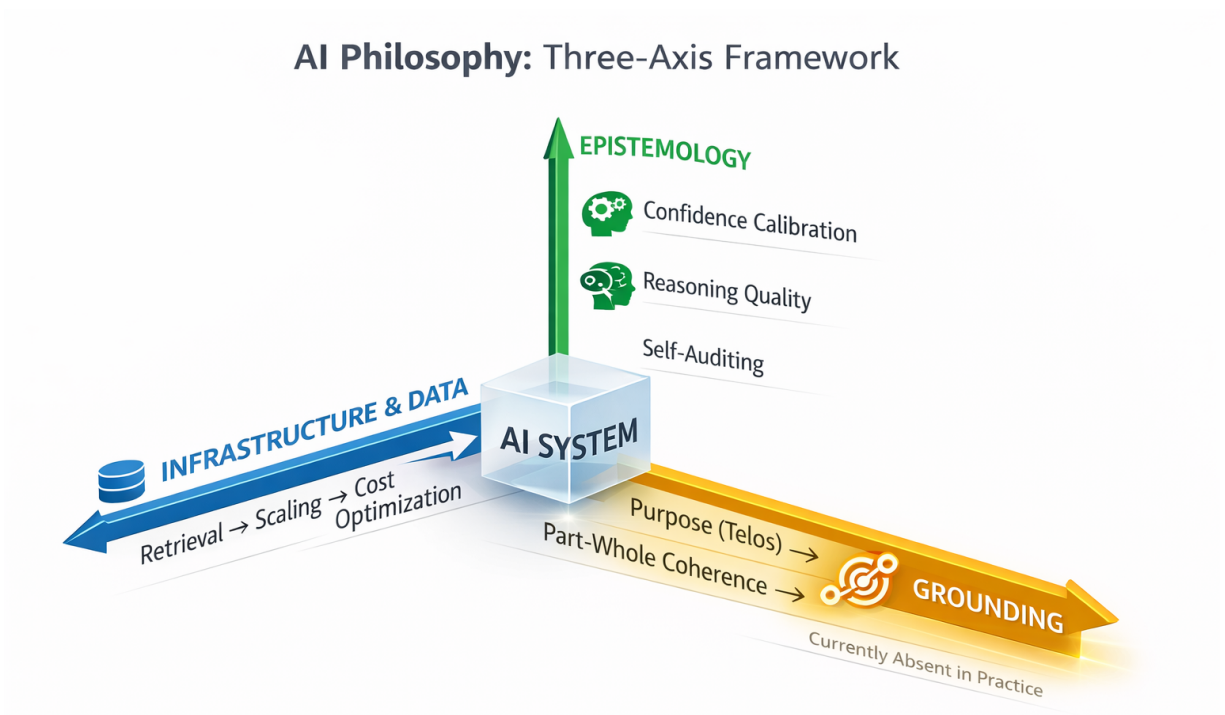


Figure 1: AI Philosophy Three-Axis Framework. The horizontal axis addresses infrastructure and data: retrieval, scaling, cost optimization. The vertical axis addresses epistemology: confidence calibration, reasoning quality, self-auditing. The grounding axis addresses teleology and mereology: purpose, part-whole coherence, integration. Current practice focuses on the horizontal, with emerging attention to the vertical. The grounding axis remains almost entirely absent in practice, explaining why systems that work in demos fail in production.

4.2 Horizontal Axis: Infrastructure and Data

The horizontal axis encompasses what most AI engineering discourse addresses: retrieval systems, vector databases, context management, embeddings, fine-tuning, prompt engineering, scaling, latency optimization, cost management. This is the infrastructure that makes AI systems function.

Work on the horizontal axis is mature. Best practices exist. Tooling is sophisticated. Organizations know how to evaluate horizontal performance:

retrieval accuracy, latency, throughput, cost per query. The engineering is real, and organizations that neglect it will fail for horizontal reasons.

The contribution of the horizontal axis is necessary foundation. Without solid infrastructure, nothing else works. A system cannot reason well about information it cannot retrieve. It cannot serve purposes if it cannot function.

The limitation is that horizontal excellence does not produce vertical or grounding success. An organization can have perfect retrieval, optimal scaling, and minimal latency while still deploying systems that are confidently wrong and serve no coherent purpose. The horizontal axis is necessary but not sufficient.

The diagnostic question for horizontal failures: Is the system failing because it lacks the right information, because retrieval is slow or inaccurate, or because infrastructure cannot scale? If yes, the intervention is horizontal. If the system has the right information, retrieves it efficiently, and scales appropriately but still fails, the problem is elsewhere.

4.3 Vertical Axis: Epistemology and Reasoning Quality

The vertical axis addresses whether systems reason well about what they receive and whether they track their own epistemic status. This includes calibrated confidence, categorical precision, the capacity for self-auditing, and corrigibility.

Current AI systems have a specific epistemic profile. They express confidence without tracking reliability. They cannot distinguish pattern frequency from evidential support. They treat all outputs as equivalent regardless of epistemic status. A retrieved fact, a statistical extrapolation, a logical inference, and a guess all emerge with similar presentation. The system does not know what it knows.

The hallucination inevitability proofs establish that this is partly architectural. Systems trained on finite data using probabilistic methods will produce outputs unsupported by their training. This cannot be engineered away. But the architectural response to inevitable hallucination is not resignation. It is building systems that know when they are on solid ground and when they are not.

Epistemic self-awareness is the capacity to track the epistemic status of outputs. A system with this capacity would distinguish:

- Retrieved information with high source reliability

- Statistical patterns with strong support in training data
- Logical inferences from provided premises
- Extrapolations beyond training distribution
- Guesses with low confidence

Current systems collapse these categories. Framework-informed systems would maintain them. The difference is architectural, not just behavioral. A system trained to express uncertainty is not the same as a system structured to track epistemic status. The former learns patterns of hedging language. The latter maintains internal representations of epistemic provenance.

The diagnostic question for vertical failures: Is the system failing because it reasons poorly, because it is confidently wrong, or because it cannot distinguish what it knows from what it is guessing? If yes, the intervention is vertical. If the system reasons well and is appropriately calibrated but still fails in production, the problem is elsewhere.

4.4 Grounding Axis: Teleology and Mereology

The grounding axis addresses two related questions: What is this system for (teleology), and how does it relate to larger wholes (mereology)?

Teleology concerns purpose. A system that does not know what it is for cannot evaluate whether its outputs serve that purpose. It optimizes for whatever metrics it is given, which may or may not track anything that matters.

The proxy-purpose gap is the divergence between what systems optimize for and what they are supposed to achieve. RLHF systems optimize for human approval ratings, which correlates with but is not identical to being genuinely helpful. Benchmark-optimized systems perform well on benchmarks, which correlates with but is not identical to production utility. The gap between proxy and purpose is where value gets lost.

Telos-constrained design makes purpose explicit and uses it to constrain optimization. Instead of treating purpose as implicit (whatever we trained the system to do), it treats purpose as a design artifact that shapes what counts as success. A system designed with teleological clarity can evaluate its own outputs against purpose, not just against proxy metrics.

Mereology concerns part-whole relationships. A system that works correctly in isolation may fail when integrated into a larger whole. The agent failure pattern, where 95% of pilots fail to reach production, is substantially a mereological failure. The parts work. The wholes do not.

Current systems are not designed to model their role within larger systems. They do not represent their own boundaries, the conditions under which their outputs are appropriate, or how downstream systems will consume what they produce. They cannot evaluate whether a locally correct response serves the larger system's purposes.

Mereological coherence is the property of parts working correctly within wholes. It requires that components be designed with awareness of the wholes they will belong to. Interface contracts specify not just what a component produces but the conditions under which outputs are appropriate for downstream use. System-level success criteria evaluate components based on their contribution to whole-system outcomes, not just local performance.

The diagnostic question for grounding failures: Is the system failing because its purpose is unclear, because it optimizes for proxies that diverge from purpose, or because it works in isolation but fails when integrated into larger systems? If yes, the intervention is grounding. These failures cannot be addressed by horizontal or vertical improvements. They require teleological clarity and mereological coherence.

4.5 How the Axes Interact

The three axes are not independent. They interact in ways that shape architectural requirements.

Horizontal success enables vertical and grounding work. A system that cannot retrieve information cannot reason about it. A system that cannot function cannot serve purposes. Infrastructure comes first chronologically, though not in importance.

Vertical improvements have limits without grounding constraints. A system can be perfectly calibrated in its confidence while still serving no coherent purpose. Epistemic self-awareness tells the system what it knows; teleological clarity tells it what matters.

Grounding provides the frame that determines what horizontal and vertical success mean. What counts as "the right information" depends on purpose. What counts as "reasoning well" depends on what the reasoning is for. Without grounding, horizontal and vertical criteria float free, optimizing for metrics that may not track what matters.

The failure rate reflects the interaction. Organizations invest heavily in horizontal infrastructure. Some are beginning to address vertical concerns. Almost none address grounding. The result is systems that retrieve information efficiently, sometimes reason about it well, but serve no clear purpose and fail when integrated into larger wholes. The grounding axis is the bottleneck.

4.6 The Framework as Diagnostic Tool

The three-axis framework provides diagnostic capacity that current approaches lack. When a system fails, the framework asks: which axis does this failure belong to?

Horizontal failures involve wrong or missing information, slow or inaccurate retrieval, scaling problems, infrastructure limitations. These failures respond to horizontal interventions: better retrieval, cleaner data, more efficient architecture.

Vertical failures involve confident wrong answers, uncalibrated outputs, reasoning errors, inability to distinguish knowledge from guessing. These failures respond to vertical interventions: calibration training, uncertainty quantification, epistemic status tracking.

Grounding failures involve systems that work correctly in isolation but fail in integration, systems that succeed at metrics while failing at purpose, systems whose outputs are locally correct but systemically destructive. These failures respond to grounding interventions: teleological clarity, mereological design, purpose-constrained optimization.

Without the framework, all failures look operational. Organizations apply operational interventions to structural problems, wonder why the problems persist, and conclude that AI is harder than expected. With the framework, organizations can match interventions to failure types, addressing structural problems with structural solutions.

5. Research Agenda: Four Directions

The framework identifies gaps. The research agenda fills them. Four directions address the vertical and grounding axes where current work is insufficient.

5.1 Overview

Each research direction addresses a core question, identifies the current gap, specifies a research program, and indicates practical implications. The directions are complementary. Progress on one enables and informs progress on others. Together, they constitute the initial program for AI Philosophy as a design discipline.

5.2 Research Direction 1: Epistemically Self-Aware Architecture

Core question: What architectural features enable systems to track the epistemic status of their outputs?

Current gap: Systems conflate pattern frequency with reliability. Confidence is a trained behavior, not a tracked property. Systems cannot distinguish what they know from what they are guessing because they lack the internal representations to maintain that distinction.

Research program:

The first task is identifying what internal representations would be required to mark epistemic distinctions. At minimum, systems would need to distinguish: retrieved information (with source reliability indicators), statistical patterns (with support strength), logical inferences (with validity tracking), extrapolations (with distance-from-training indicators), and guesses (with low-confidence markers). Whether current architectures can support such representations, or whether structural changes are required, is an open question.

The second task is determining what can be achieved through training versus what requires architectural change. Some epistemic behaviors can be trained. But training produces learned behaviors, not structural capacities. A system trained to express uncertainty may learn hedging language patterns rather than genuine epistemic tracking. The research must distinguish superficial behavioral compliance from architectural epistemic awareness.

The third task is developing evaluation criteria that go beyond calibration curves. Current calibration metrics ask whether confidence levels match accuracy rates in aggregate. Epistemic self-awareness requires more: Can the system distinguish why it is confident in different cases? Can it identify when it is extrapolating beyond training? Can it flag when a query exceeds its competence? Evaluation criteria for these capacities do not yet exist.

Practical implications: Systems with epistemic self-awareness enable confidence-gated workflows where system behavior changes based on epistemic status. High-confidence outputs proceed automatically; low-confidence outputs route to human review. Such systems also enable epistemic status tagging, where outputs carry metadata distinguishing claim types for downstream processing. For organizations, the implication is reduced liability from confidently wrong outputs and increased trust from appropriately calibrated uncertainty.

5.3 Research Direction 2: Telos-Constrained Optimization

Core question: How can purpose function as an architectural constraint rather than a training target?

Current gap: Optimization toward proxies produces proxy-satisfying behavior that diverges from purpose. Systems succeed at benchmarks while failing at production utility. RLHF produces systems that satisfy raters rather than systems that are genuinely helpful. The gap between proxy and purpose is where value disappears.

Research program:

The first task is developing the formal distinction between telos-as-target and telos-as-constraint. Current systems optimize toward objectives (targets). An alternative is optimizing within constraints set by purpose. The constraint approach asks not "how do we maximize this metric?" but "what optimizations are permissible given this purpose?" The formal properties of constraint-based versus target-based optimization require development.

The second task is identifying architectural mechanisms that encode constraints rather than objectives. Objective functions are well understood. Constraint encoding is less developed. How does purpose get represented such that it bounds optimization rather than directing it? What architectural features would enable a system to recognize when optimization is diverging from purpose?

The third task is developing methods for purpose representation that enable system self-evaluation. A system with teleological clarity can evaluate its outputs against purpose, not just against proxy metrics. This requires that purpose be represented in a form the system can use. Natural language purpose statements are a start, but machine-interpretable purpose representations that enable automated evaluation remain undeveloped.

The fourth task is proxy-purpose gap detection: methods for identifying when optimization is succeeding at proxies while failing at purpose. This requires both purpose representation and comparison mechanisms that can flag divergence before deployment failure reveals it.

Practical implications: Telos-constrained design produces systems that succeed at what matters, not just at what is measured. Purpose specification becomes a design artifact, created before architecture decisions and used to constrain them. Evaluation criteria derive from purpose, not just from benchmarks. Organizations gain reduced waste from systems that succeed at the wrong things and faster time to value from systems designed with clear purpose from the start.

5.4 Research Direction 3: Mereological Coherence by Design

Core question: What architectural features enable systems to function as parts of larger wholes?

Current gap: Systems do not represent their own boundaries, model downstream use, or evaluate outputs against system-level success criteria. Components designed in isolation fail when integrated. The agent failure pattern is substantially a mereological failure pattern.

Research program:

The first task is identifying what representations would enable a system to model its role within a larger system. This includes boundary representations (what the component takes in, what it produces, what assumptions it makes), dependency representations (what the component relies on, what relies on it), and context representations (the conditions under which the component operates correctly).

The second task is developing interface contracts that specify not just what a component produces but the conditions under which outputs are appropriate for downstream use. Current interfaces specify data formats. Mereological

interfaces would specify use conditions: this output is appropriate for decision support but not for automated action; this output assumes context that must be verified; this output degrades under conditions the downstream system should monitor.

The third task is developing memory architectures that track context of use across interactions. Components that remember how their outputs were used can learn mereological constraints. If an output intended for one purpose is consistently used for another, that is mereological information the system could track and flag.

The fourth task is developing evaluation criteria for part-whole coherence. Current evaluation assesses components in isolation. Mereological evaluation assesses contribution to system outcomes. A component that scores well in isolation but degrades system performance when integrated fails the mereological criterion even if it passes the component criterion.

Practical implications: Systems designed for mereological coherence integrate successfully at higher rates. The agent failure pattern diminishes because agents are designed with awareness of the wholes they serve. Integration testing becomes mereological validation: do the parts work as parts, not just as isolated components? Organizations gain reduced integration failures and systems that remain useful when scaled beyond initial deployment.

5.5 Research Direction 4: The Origination-Derivation Boundary as Architectural Feature

Core question: Can architecture encode the boundaries of derivative competence?

Current gap: Current systems handle limits through refusal training, a behavioral intervention that teaches systems to decline certain requests. But refusal training does not give systems structural awareness of what they can and cannot supply. It gives them patterns for declining requests that match training examples of inappropriate requests.

The deeper issue is that AI systems operate derivatively. They process and recombine existing information. They do not originate. Some queries require origination: novel judgment, genuine creativity, semantic understanding that goes beyond pattern matching. Systems that cannot distinguish queries within

their derivative competence from queries requiring origination will attempt both with equal confidence.

Research program:

The first task is developing criteria for identifying queries that exceed derivative competence. What features of a query indicate that it requires origination the system cannot supply? This connects to work on semantic unity, inferential normativity, and truth-aptness from philosophy of mind and logic. Queries requiring genuine understanding, as opposed to pattern-matched responses that mimic understanding, exceed derivative competence.

The second task is developing architectural mechanisms for boundary recognition that go beyond behavioral refusal. A system with structural awareness of its limits would not merely decline requests matching trained refusal patterns. It would recognize when a query's features indicate requirements it cannot meet, even for novel query types not represented in refusal training.

The third task is connecting this to deployment patterns. How should human-AI interaction change when systems have boundary awareness? If a system can flag "this query exceeds my competence," what should happen? The research must develop not just the architectural capacity but the interaction patterns that make use of it.

Practical implications: Systems with boundary awareness are used appropriately by design, not just by user discipline. The division of labor between human and AI becomes architecturally encoded rather than relying on users to know the limits. Organizations gain reduced misapplication, clearer human-AI collaboration patterns, and systems that direct attention to human judgment where human judgment is required.

5.6 Cross-Cutting Questions

The four directions interact. Progress on epistemic self-awareness informs boundary recognition: knowing what you know helps identify where you don't know. Teleological clarity informs mereological coherence: knowing what a component is for clarifies how it should relate to wholes. Mereological awareness informs epistemic status: a component that models its role in larger systems can better assess whether its outputs serve system-level purposes.

Several questions span the directions:

What existing research programs connect to these directions, and where do they fall short? The uncertainty quantification literature connects to Direction 1 but lacks architectural integration. Alignment research connects to Direction 2 but assumes rather than establishes purpose. Systems engineering connects to Direction 3 but lacks philosophical foundations for part-whole relationships. Philosophy of mind connects to Direction 4 but lacks application to AI architecture.

What empirical work would test whether architectural changes produce intended effects? Each direction implies testable predictions. Systems with epistemic self-awareness should show different calibration profiles than systems merely trained to express uncertainty. Systems with teleological clarity should show smaller proxy-purpose gaps. Systems designed for mereological coherence should show higher integration success rates. Systems with boundary awareness should show more appropriate use patterns. Empirical validation requires operationalizing these predictions and testing them.

How do we distinguish genuine improvement from more sophisticated proxy-satisfaction? Each direction faces the risk that interventions produce appearance of improvement rather than substance. Systems might learn to mimic epistemic awareness without achieving it. Purpose specification might become another proxy. Mereological metrics might be gamed. Boundary recognition might become stylized refusal. The research must develop methods for distinguishing genuine architectural capacity from behavioral mimicry.

6. Practical Application: From Framework to Architecture

The framework provides diagnostic capacity. The research agenda provides directions for development. This section demonstrates how the framework translates into architectural decisions that organizations can make now, without waiting for the research agenda to complete.

6.1 The Implementation Insight

Organizations are already investing heavily in AI infrastructure. They are building retrieval systems, developing metadata schemas, constructing

knowledge graphs, engineering context management pipelines. This investment is substantial and ongoing.

The insight is that the same infrastructure could carry vertical and grounding axis information with relatively modest extension. The investment has been made. The question is whether it produces horizontal-only returns or full-framework returns.

Current metadata schemas optimize for retrieval: source, date, domain, document type, entities mentioned, relevance scores. This serves the horizontal axis. The same schemas could carry epistemic metadata (claim type, evidential basis, reliability indicators), teleological metadata (purpose, appropriate use conditions, misuse indicators), and mereological metadata (part-whole relationships, dependencies, integration constraints). The infrastructure investment organizations have already made becomes the foundation for framework-informed architecture.

6.2 Architectural Decision Points

The framework applies at every layer of the AI stack. At each layer, decisions currently made on horizontal criteria alone could be made on three-axis criteria.

Data layer. Current practice tags data for retrieval efficiency. Framework-informed practice adds epistemic tagging (is this a fact, an inference, an opinion, a disputed claim?), teleological tagging (what decisions does this information support, under what conditions is it applicable?), and mereological tagging (what larger structures does this belong to, what depends on it?). The data layer becomes not just a retrieval system but an epistemic and purposive resource.

Model selection and configuration. Current practice selects models based on capability, cost, and latency. Framework-informed practice adds: What epistemic characteristics does this task require? What is the telos of this component? What whole will it serve? A model appropriate for creative generation may be inappropriate for factual retrieval, not because it lacks capability but because its epistemic profile does not match the task's requirements.

Prompt and context engineering. Current practice optimizes for relevance and recency. Framework-informed practice adds epistemic provenance (what is the epistemic status of this context?), purpose-alignment (does this context serve the purpose at hand?), and mereological relationships (how do these context

elements relate to each other and to the larger query?). Context injection becomes purposive, not just relevant.

Output processing. Current practice applies safety filters, formatting, and sometimes fact-checking. Framework-informed practice adds epistemic status assignment (what kind of claim is this output making?), purpose-alignment validation (does this output serve the stated purpose?), and interface contract enforcement (does this output meet the conditions for downstream use?). Output processing becomes a checkpoint for vertical and grounding compliance, not just safety and formatting.

Orchestration and agent design. Current practice focuses on task decomposition, tool selection, and workflow management. Framework-informed practice adds mereological coherence between components (do these parts work as a whole?), telos-consistency across the workflow (does each step serve the overarching purpose?), and epistemic status propagation (how does uncertainty compound through the chain?). Agent orchestration becomes architecturally aware of part-whole relationships and purpose.

Evaluation and monitoring. Current practice measures accuracy, latency, user satisfaction, and benchmark performance. Framework-informed practice adds proxy-purpose gap tracking (is the system succeeding at metrics while failing at purpose?), part-whole coherence metrics (do components work as well integrated as they do in isolation?), and epistemic calibration monitoring (is expressed confidence tracking actual reliability over time?). Evaluation becomes multi-axis, not just performance-focused.

Human-AI interface design. Current practice optimizes user experience, provides explanations, and enables feedback. Framework-informed practice adds epistemic status communication (showing users when the system is confident versus uncertain and why), purpose-transparency (making clear what the system is for and what lies outside its scope), and boundary visibility (indicating where human judgment is required). The interface becomes a site of appropriate trust calibration, not just usability.

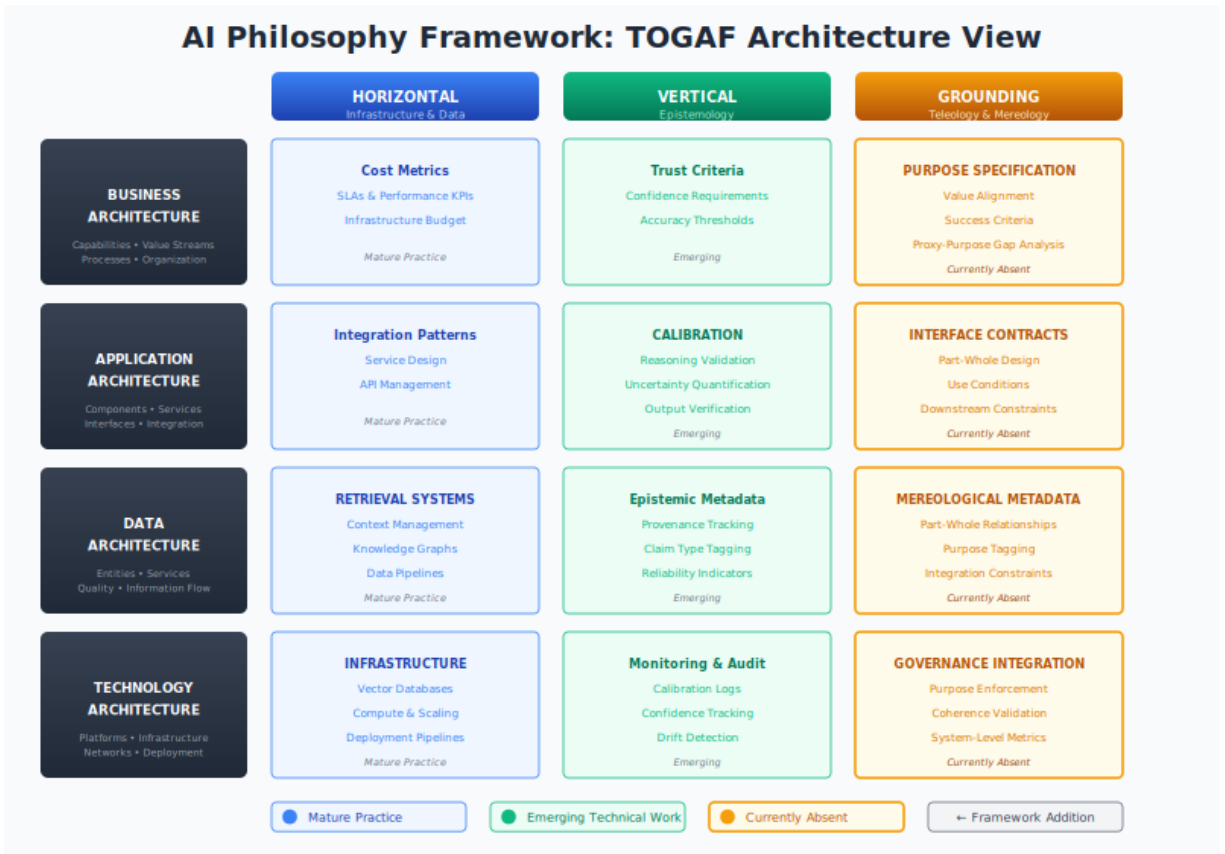


Figure 2: Framework Mapping to Enterprise Architecture. The three axes map to standard enterprise architecture layers (following TOGAF conventions). Each cell indicates framework-informed considerations for that layer. The horizontal axis (blue) shows mature practice across all layers. The vertical axis (green) shows emerging practice, primarily at application and data layers. The grounding axis (gold) shows the gap: purpose specification, value alignment, and success criteria remain largely absent from current architectural practice. The gold column represents this paper's primary contribution.

6.3 Implementation Pathway

Organizations need not transform their entire AI practice at once. The framework supports graduated adoption.

Level 1: Diagnostic application. Use the framework to analyze existing failures. When a pilot fails or a deployed system underperforms, ask: which axis does this failure belong to? Is it horizontal (infrastructure), vertical (epistemology), or grounding (teleology, mereology)? This requires no new

builds, only a new diagnostic lens. The benefit is accurate diagnosis, which enables appropriate intervention. Organizations that apply operational interventions to structural problems waste resources. Organizations that can identify structural problems apply structural solutions.

Level 2: Design integration. Incorporate framework elements into new system design. Before architecture decisions, specify purpose explicitly. Conduct proxy-purpose gap analysis: under what conditions might this system succeed at metrics while failing at purpose? Define epistemic requirements: what kinds of claims will this system make, and how will epistemic status be tracked? Specify part-whole relationships: what larger systems will this integrate into, and what interface contracts govern appropriate use? This changes how systems get built, introducing vertical and grounding considerations at design time rather than discovering their absence at deployment.

Level 3: Architectural embedding. Build the framework into system architecture itself. Systems track their own epistemic status as a structural feature. Purpose constrains optimization architecturally, not just aspirationally. Part-whole relationships are represented and enforced. This is the full vision of framework-informed architecture. It requires the research agenda to advance but can be approached incrementally as research matures.

Most organizations should enter at Level 1. The framework earns credibility by explaining failures that current approaches cannot explain. Success at Level 1 builds organizational understanding and creates demand for Level 2. Success at Level 2 creates foundation for Level 3 as research and tooling mature.

6.4 What This Paper Does Not Provide

This paper establishes the framework and demonstrates its architectural relevance. It does not provide:

Detailed implementation specifications. Reference architectures, code patterns, and integration guides require development beyond this paper's scope. They are appropriate for follow-on work once the framework is established.

Comprehensive tooling. Assessment instruments, diagnostic checklists, and monitoring dashboards operationalize the framework for practitioners. These require development and validation.

Case studies. Detailed analyses of framework application to specific failures and successes would strengthen the empirical case. They will emerge as organizations adopt the framework and document their experience.

The paper provides foundations. The practical apparatus builds on those foundations. The sequence is appropriate: establish the framework, then develop the tools that operationalize it.

7. Objections and Responses

The framework will face objections. Addressing them directly clarifies what the framework claims and does not claim.

7.1 "This adds complexity without proven ROI"

The objection notes that the framework introduces new concepts and criteria. Organizations already struggle with AI complexity. Adding philosophical dimensions seems to compound the problem without demonstrated return.

The response: The 95% failure rate is the current ROI. Organizations already pay the cost of structural failures in failed pilots, abandoned initiatives, and systems that work in demos but not in production. The framework does not add complexity to a simple situation. It provides vocabulary for complexity that already exists but cannot currently be articulated.

Diagnostic application (Level 1) requires no new investment, only new analysis. If the framework explains failures that current approaches cannot explain, it has demonstrated value.

7.2 "Philosophy is too abstract for engineering decisions"

The objection holds that philosophical concepts like teleology and mereology belong in academic discourse, not in architectural decision-making. Engineering requires concrete specifications, not abstract frameworks.

The response: Every framework is abstract until applied. The framework translates to specific architectural decision points, as Section 6 demonstrates. Every layer of the AI stack has framework-informed alternatives to current

practice. The abstraction provides diagnostic power; the architectural application provides engineering decisions.

Furthermore, the objection itself rests on a philosophical position: that engineering can proceed without attention to purpose and part-whole relationships. That position has produced 95% failure rates. The claim that philosophy is impractical is a philosophical claim, and it is empirically falsified by the failure data. Philosophy is already present in AI architecture; the question is whether it is present explicitly and well or implicitly and poorly.

7.3 "We can solve these problems with better data and bigger models"

The objection holds that the failures documented are transient. As models improve and data quality increases, the problems will resolve. Scaling solves everything eventually.

The response: The hallucination inevitability proofs establish architectural limits that scaling cannot overcome. Larger models trained on more data will still hallucinate because hallucination emerges from the mathematical structure of how these systems work, not from insufficient scale. Better data improves horizontal performance. Bigger models improve capability. Neither addresses the vertical question (does the system know what it knows?) or the grounding questions (what is it for, and how does it fit?).

The MIT NANDA report found that the largest enterprises, with the most resources for data and models, had the worst conversion rates from pilot to production. Scale is not solving the problem. The problem is structural, and structural problems require structural solutions.

7.4 "Existing alignment and safety research already covers this"

The objection holds that the concerns raised, particularly around purpose and values, are addressed by the AI alignment and safety research communities. The framework duplicates existing work.

The response: Alignment research asks how to encode values in AI systems given that we are building them. It presupposes purpose and asks how to align behavior with values within that purpose. The framework asks the prior question: what is this system for? How does purpose get established, represented, and used to constrain design?

Safety research asks how to prevent AI systems from causing harm. It addresses failure modes but not success modes. A system can be safe (causing no harm) while still failing in production (delivering no value). Safety is necessary but not sufficient.

The framework complements alignment and safety by providing architectural foundations they presuppose. Aligned, safe systems still fail at 95% rates because alignment and safety do not address the grounding axis. The framework fills the gap that alignment and safety leave open.

7.5 "The business case is speculative - show me case studies"

The objection demands empirical validation before adoption. The framework may be intellectually coherent but practically unvalidated.

The response: The failure case is documented. The MIT NANDA report, the S&P Global data, the BCG analysis, and the Gartner predictions constitute substantial empirical evidence of a problem. The framework explains that evidence in ways current approaches do not.

Case studies of framework application will emerge from organizations that adopt it. This paper establishes foundations; empirical validation follows adoption. The alternative is continued investment in approaches with documented 95% failure rates.

7.6 "Our organization isn't ready for this level of philosophical sophistication"

The objection holds that even if the framework is valid, most organizations lack the philosophical background to apply it. It requires expertise organizations do not have.

The response: The framework scales to organizational readiness. Level 1 adoption requires only willingness to analyze failures using a new diagnostic lens. The concepts, once named, are intuitive. Organizations already experience grounding failures; they lack vocabulary to identify them.

"What is this system for?" is not a philosophically sophisticated question. "Does this component work as part of the larger system?" is not arcane. The framework makes explicit what practitioners already implicitly understand.

8. Conclusion: The Discipline That Was Missing

8.1 What the 95% Failure Rate Reveals

The failure rate is too consistent to be operational bad luck. When the same patterns appear across industries, vendors, and organizational types, the cause is structural. Structural causes require structural solutions.

The structural cause is the absence of grounding. Organizations invest in infrastructure (horizontal axis) and are beginning to address reasoning quality (vertical axis). Almost none address purpose (teleology) or part-whole relationships (mereology). Systems are built without clarity about what they are for and without design for integration into larger wholes. They work in demos and fail in production because demos do not test grounding; production does.

The 95% failure rate is what happens when an entire industry optimizes along one axis while ignoring two others. The framework makes the ignored axes visible. Making them visible is the first step toward addressing them.

8.2 What AI Philosophy Provides

AI Philosophy, as this paper establishes it, is a design discipline. It provides:

A framework for diagnosis. The three-axis model enables accurate failure attribution. Horizontal failures get horizontal solutions. Vertical failures get vertical solutions. Grounding failures get grounding solutions. Without the framework, all failures look operational, and organizations apply operational interventions to structural problems.

A vocabulary for articulation. Problems that cannot be named cannot be addressed. The framework provides vocabulary for grounding concerns that current architectural discourse lacks. Teleology, mereology, epistemic self-awareness, proxy-purpose gaps, interface contracts, part-whole coherence: these terms name realities that organizations experience but cannot currently articulate.

A research agenda for development. The four directions, epistemic self-awareness, telos-constrained optimization, mereological coherence, and origination-derivation boundaries, provide systematic paths for advancing the discipline. The agenda is specific enough to guide research and open enough to allow diverse contributions.

Practical application across the stack. Every architectural layer has framework-informed alternatives to current practice. The framework is not merely theoretical. It translates to engineering decisions at data, model, prompt, output, orchestration, evaluation, and interface layers.

A pathway for adoption. Organizations can enter at diagnostic application (Level 1), advance to design integration (Level 2), and eventually achieve architectural embedding (Level 3). The pathway respects organizational readiness while enabling progressive development.

8.3 The Business Proposition

The proposition is direct. Organizations that address only horizontal concerns will continue experiencing high failure rates. The evidence is clear: infrastructure excellence does not produce production success. Something else is required.

The framework identifies what else is required: vertical and grounding axis attention. Organizations that develop this capacity will diagnose failures accurately, design systems with purpose and part-whole coherence, and achieve higher success rates. The competitive advantage accrues to early movers. The cost of inaction is continued investment in approaches that fail 95% of the time.

8.4 What Comes Next

This paper establishes foundations. What comes next is construction.

Empirical validation. Organizations applying the framework will generate case studies. Diagnostic application to existing failures will test whether the framework explains what current approaches cannot. Design integration will test whether framework-informed systems succeed at higher rates. The empirical record will develop through adoption.

Practical tooling. Reference architectures, assessment instruments, diagnostic checklists, and monitoring dashboards operationalize the framework for practitioners. These require development beyond what a single paper can provide.

Research execution. The four directions require sustained investigation. Epistemic self-awareness, telos-constrained optimization, mereological coherence, and origination-derivation boundaries each represent substantial

research programs. Contributions from multiple researchers across multiple institutions will advance the agenda faster than any single effort.

Community development. AI Philosophy as a discipline requires a community: researchers developing the theory, practitioners applying it, organizations testing it, and venues for sharing results. The discipline is open for contribution. This paper provides foundations; the community builds the edifice.

8.5 The Question That Drives This Work

The question is simple: When the infrastructure hums, the data is clean, and the reasoning is sound, what is this system for, and how does it fit?

Current AI practice has no systematic answer. Systems are built to function, optimized to perform, and deployed to see what happens. When they fail, as they do 95% of the time, the failures are attributed to operations. The structural questions go unasked.

AI Philosophy is the discipline that asks the structural questions. It takes purpose and part-whole relationships seriously as engineering requirements. It provides the framework that connects business problems to structural causes to architectural solutions.

The 95% failure rate is what happens when the questions go unasked. This paper provides the framework for asking them. What comes next depends on whether organizations choose to ask.

References

Banerjee, S., Agarwal, A., & Singla, S. (2024). LLMs will always hallucinate, and we need to live with this. arXiv preprint arXiv:2409.05746. <https://arxiv.org/abs/2409.05746>

Boston Consulting Group. (2024). AI adoption and value creation: Global executive survey.

Challapally, A., Pease, C., Raskar, R., & Chari, P. (2025). The GenAI divide: State of AI in business 2025. MIT Project NANDA. https://mlq.ai/media/quarterly_decks/v0.1_State_of_AI_in_Business_2025_Report.pdf

Cotnoir, A. J., & Varzi, A. C. (2021). *Mereology*. Oxford University Press. <https://global.oup.com/academic/product/mereology-9780198749004>

Gabriel, I., & Keeling, G. (2025). A matter of principle? AI alignment as the fair treatment of claims. *Philosophical Studies*, 182(7), 1951-1973. <https://doi.org/10.1007/s11098-025-02300-4>

Gartner. (2025). *Predicts 2025: AI and the future of work*.

Kalai, A. T., Zhang, E., Nachum, O., & Vempala, S. S. (2025). Why language models hallucinate. OpenAI. <https://openai.com/index/why-language-models-hallucinate/>

McQueen, K. J., & Tsuchiya, N. (2023). When do parts form wholes? Integrated information as the restriction on mereological composition. *Neuroscience of Consciousness*, 2023(1), niad013. <https://doi.org/10.1093/nc/niad013>

S&P Global. (2024). *Enterprise AI initiative outcomes report*. [Industry report].

Schrage, M., & Kiron, D. (2025, January 16). Philosophy eats AI. MIT Sloan Management Review. <https://sloanreview.mit.edu/article/philosophy-eats-ai/>

Xu, Z., Jain, S., & Kankanhalli, M. (2024). Hallucination is inevitable: An innate limitation of large language models. arXiv preprint arXiv:2401.11817. <https://arxiv.org/abs/2401.11817>

Zhi-Xuan, T., Carroll, M., Franklin, M., & Ashton, H. (2025). Beyond preferences in AI alignment. *Philosophical Studies*, 182(7), 1813-1863. <https://doi.org/10.1007/s11098-024-02249-w>

Appendix A: Glossary of Key Terms

Horizontal axis: The dimension of AI architecture addressing infrastructure, data, retrieval, scaling, and cost optimization. Necessary foundation for AI systems; not sufficient for production success.

Vertical axis: The dimension addressing epistemology and reasoning quality: calibrated confidence, categorical precision, self-auditing, corrigibility. Determines whether outputs are trustworthy and whether systems know what they know.

Grounding axis: The dimension addressing teleology (purpose) and mereology (part-whole relationships). Determines whether systems serve coherent purposes and function correctly within larger wholes.

Teleology / Telos: The study of purpose. In AI architecture, concerns what a system is for, not just what it does. Without telos, optimization targets proxies that may not track what matters.

Mereology: The study of parts and wholes. In AI architecture, concerns how components relate to systems they belong to. Without mereological awareness, locally correct outputs produce systemically incoherent effects.

Proxy-purpose gap: The divergence between what a system optimizes for (the proxy) and what it is supposed to achieve (the purpose). Systems can succeed at proxies while failing at purpose.

Epistemic self-awareness: A system's capacity to track the epistemic status of its outputs: distinguishing retrieved facts from inferences, statistical patterns from extrapolations, knowledge from guessing.

Interface contracts: Output specifications including not just content and format but conditions of appropriate downstream use. Enable mereological coherence by specifying how outputs should be consumed.

Part-whole coherence: The property of system components working correctly when integrated into larger systems. Components that perform well in isolation may fail to exhibit part-whole coherence.

Origination-derivation boundary: The distinction between what AI systems can derive from existing information and what requires human origination. Some queries exceed derivative competence.

Appendix B: Framework Diagnostic Reference

When a system fails, ask: which axis does this failure belong to?

Horizontal failure indicators: - Wrong or missing information retrieved - Slow or inaccurate retrieval - Scaling problems under load - Infrastructure limitations - Data quality issues

Horizontal interventions: Better retrieval systems, cleaner data, more efficient architecture, improved scaling.

Vertical failure indicators: - Confident wrong answers - Uncalibrated confidence (overconfident or underconfident) - Reasoning errors given correct inputs - Inability to distinguish knowledge from guessing - Hallucination on topics within scope

Vertical interventions: Calibration training, uncertainty quantification, epistemic status tracking, reasoning validation.

Grounding failure indicators: - System works in isolation but fails in integration - System succeeds at metrics but fails at purpose - Outputs are locally correct but systemically problematic - Purpose is unclear or contested - System optimizes for proxies that diverge from intent

Grounding interventions: Purpose specification, teleological clarity, mereological design, interface contracts, system-level evaluation criteria.

If horizontal and vertical indicators are absent but failure persists, the failure is almost certainly grounding.

Appendix C: Research Direction Summary

Direction	Core Question	Current Gap	Primary Output
Epistemic Self-Awareness	How can systems track epistemic status of outputs?	Confidence is trained behavior, not tracked property	Architectures that know what they know
Telos-Constrained Optimization	How can purpose constrain rather than merely target?	Proxy-purpose gaps produce metric success with purpose failure	Systems that optimize for purpose, not just metrics
Mereological Coherence	How can systems function as parts of wholes?	No architectural modeling of part-whole relationships	Integration-ready components with interface contracts
Origination-Derivation Boundary	How can architecture encode competence limits?	Refusal training is behavioral, not structural	Appropriate use patterns built into architecture

This paper establishes AI Philosophy as a design discipline. The framework, research agenda, and practical applications are offered for development by the broader community. Correspondence and collaboration inquiries may be directed to the author.